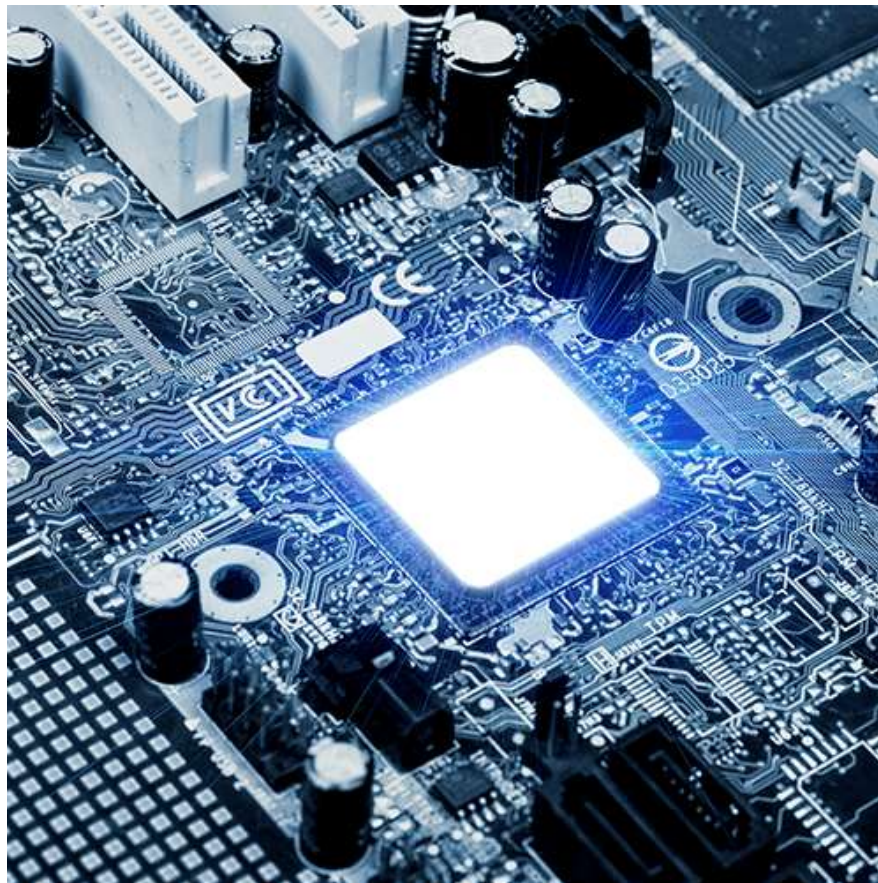


Embedded Linux Systems & the Yocto Project

HOMEPAGE ([HTTPS://WITEKIO.COM/](https://witekio.com/)) > EMBEDDED LINUX SYSTEMS & THE YOCTO PROJECT



Linux has come a long way since it saw the light of day more than 20 years ago, and is by far the most scalable operating system to exist today. From servers to desktops to embedded systems with low memory footprints, it is now hard to miss. Creating a new product today often requires the choice of an embedded operating system. For example, Google chose the Linux kernel as a core component for its phone/tablet Android operating system, increasing its overall popularity in different markets.

Historically, Linux was always considered complex and non-user friendly, only accessible to those with very specific low-level embedded software expertise. Over the past few years, this trend has changed drastically, thanks to the efforts of various communities. While building a Linux embedded system in the past required the development of custom scripts, putting together different packages, and struggling to integrate it all together, this effort is now much more streamlined.

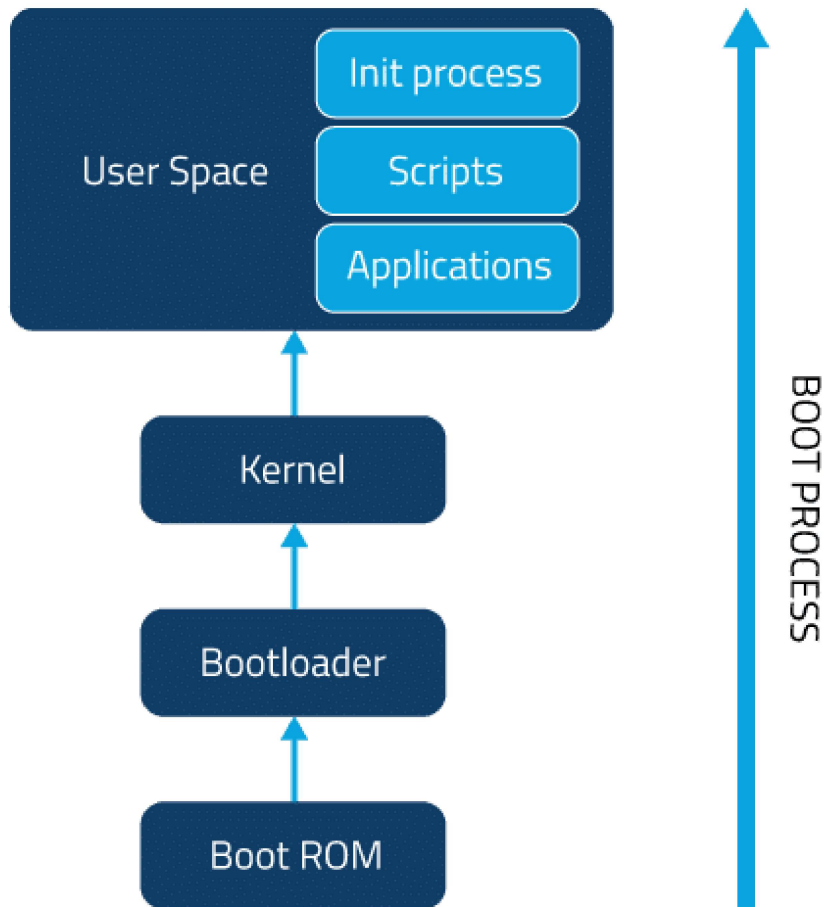
This article will attempt to shed some light on the ease of this process.

The four components of an embedded Linux system

Before delving into the details of how to build an embedded Linux system, understanding its different components from a high-level perspective is necessary. A good way to separate these pieces and help overall comprehension is to represent them through a typical embedded system boot process, as seen in the diagram below.

When powering up a device (such as a typical phone), multiple stages of execution occur before applications can be loaded and presented to the end-user. These are introduced below





1) Boot ROM

This piece of software is physically located within read-only memory inside the System-on-Chip itself and is comparable to the BIOS of a regular PC. Every vendor has its own unique flavor, which typically boils down to the same functionality: loading a further boot stage, which is the main bootloader. It cannot be modified but its execution usually can be impacted by the use of external signals such as boot pins, allowing a particular media to be selected for boot-up – it could be SD Card, eMMC, NAND flash, or serial for instance. Some boot ROMs also offer specific features such as preventing non-signed pieces of code from being executed next, hardening overall device security.

Witekio's Cookies Policy (<https://witekio.com/cookies-policy/>) - Privacy policy (<https://witekio.com/privacy-policy/>) -

Legal notices (<https://witekio.com/legal-notices/>) -

The bootloader often requires two stages of execution in itself. When the boot ROM runs, the device RAM is not initialized, therefore the first role of the bootloader is to initialize the main RAM – this piece of code typically runs from internal SRAM. The second stage runs from main RAM with the ultimate goal of running the kernel by loading it from a particular configurable media, including from distant machines (via TFTP) which is especially useful for debugging purposes. Most bootloaders these days include advanced functionality allowing a developer to flash kernels onto eMMC/NAND/NOR, and validate multiple device components such as the on-board RAM, I2C/SPI peripherals, etc. Another important aspect of bootloaders, if desired, is to handle power-on self-testing, allowing device integrity validation prior to running the main operating system.

3) Linux Kernel

The heart of the embedded Linux system – the Linux kernel – is loaded and executed by the bootloader. It is responsible for initializing all hardware peripherals, servicing interrupts, scheduling all tasks running in the system, and much more. It is the glue between user space (mainly consisting of applications) and the circuitry and is key to allowing programs to be device-agnostic and run on multiple pieces of equipment with little to no modification.

4) Root file system

One of the main tasks of the Linux kernel is to mount the so-called root file system, which contains all the applications, scripts, and libraries available in the system. Generating a root file system manually is very challenging, may it be because of dependencies or standards to follow – which is why tools exist to automatically package it. This is undoubtedly the area that evolved the most in the last few years, with multiple communities working jointly to create build systems enabling developers to select, build, and integrate packages in an easy and reproducible fashion.

GET IN TOUCH TO FIND OUT MORE ABOUT OUR YOCTO DEVELOPMENT SERVICES

Contact us

Embedded Linux build system: the Yocto Project

As said earlier, multiple-build systems have existed over the years for embedded Linux distributions. Tools such as buildroot, OpenEmbedded, LTIB were all meant to achieve a similar purpose: automatically package a root file system for developers, with the option of building a bootloader and kernel for your target as well. This trend has changed in a positive way, with multiple silicon vendors converging to a common tool for this purpose: this is where *Yocto* comes into play.

The Yocto Project (<https://www.yoctoproject.org/>) is an open-source collaboration project providing templates, tools, and methods to help you create custom Linux-based systems for embedded products regardless of hardware architecture. The main advantage of this build system is its concept of recipes, which are simple files describing where to get a particular package, how to build it, and where to install it in the resulting file system hierarchy. Most of the packages needed by developers already exist as part of this project – if not, adding a recipe for one is a straightforward process.



Here is a simple example recipe for a Hello World application which is part of the Yocto project itself. (https://witekio.com) As you can see below, it gives a brief description of the package and specifies its license, a pointer to the source code online, the appropriate revision to checkout, and hints on how to build it (this particular example is autotools-based).

```
#
# This file was derived from the 'Hello World!' example recipe in the
# Yocto Project Development Manual.
#
DESCRIPTION = "Simple helloworld application"
SECTION = "examples"
DEPENDS = ""
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://LICENSE;md5=96af5705d6f64a88e035781ef00e98a8"
FILESEXTRAPATHS_prepend := "${THISDIR}/${PN}-${PV}:"
SRCREV = "4ec84957442182f36351f94c0626560ec3ffd487"
SRC_URI = "git://github.com/DynamicDevices/bbexample.git"
S = "${WORKDIR}/git"
inherit autotools
# Do not enable parallel make
PARALLEL_MAKE = ""
```

The initial step of building an embedded Linux distribution with Yocto is to select the device we are targeting. The reason for this is that Yocto will need to select the appropriate packages for the bootloader and kernel to be built, as well as specific libraries which only match the system the resulting binaries will run on top of, such as GPU libraries for instance. Luckily for us, this does not require much work at all; only a few command lines are required for device selection as well as bootloader, kernel, and root file system generation.

6 Steps to Build a Yocto BSP for Your Embedded

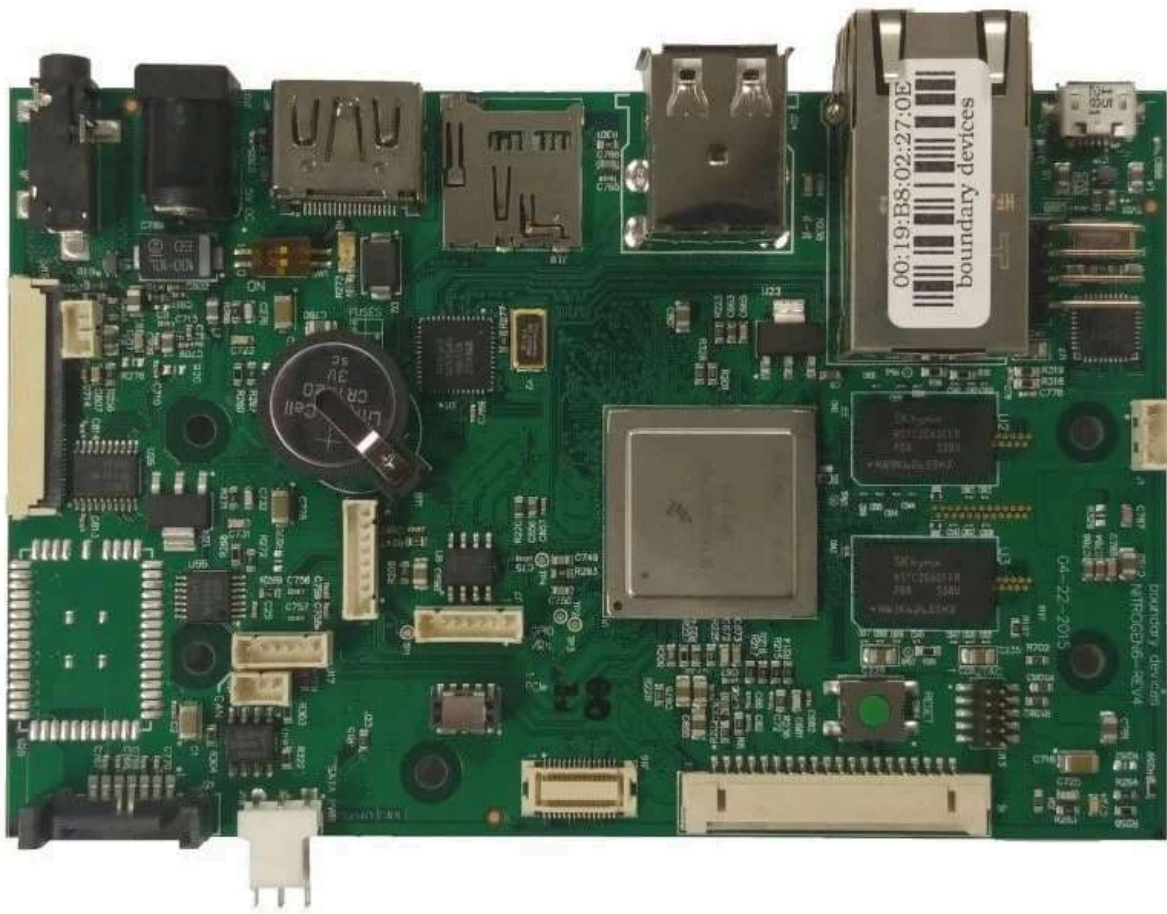
Witekio's cookie policy (https://witekio.com/cookies-policy/) - Privacy policy (https://witekio.com/privacy-policy/) -

Legal notices (https://witekio.com/legal-notices/) -



Linux System

The goal of this chapter is to go through the steps required in order to build a Yocto-based BSP and flash the resulting images onto an SD Card; Boundary Devices' Nitrogen6X target platform was chosen for this exercise. In general, all NXP's i.MX 6 board support packages rely on Yocto for generating the bootloader (u-boot) to be loaded by the internal i.MX 6 ROM code, the Linux kernel, and the root filesystem containing all libraries, middleware, tools and other applications running in user-space.



When installing Yocto for the first time, the following set of packages need to be installed on your development host machine (Ubuntu 16.04 being recommended):

<https://docs.yoctoproject.org/current/ref-manual/system-requirements.html#required-packages-for-the-build-host> (<https://docs.yoctoproject.org/current/ref-manual/system-requirements.html#required-packages-for-the-build-host>)

2) The repo utility

The *repo* utility is a small tool developed by Google on top of *Git* which role is to manage multiple Git repositories – it is also necessary for Android platform development. To install it in your development environment, issue the following commands:

```
$ mkdir ~/bin
$ cd ~/bin
$ curl http://commondatastorage.googleapis.com/git-repo-downloads/repo > repo
$ chmod a+x repo
```

3) Installation

The next step is to download and setup the BSP itself. The repo tool will be used to automatically clone the various Git repositories needed for development. To do so, create a directory for BSP storage and initialize it with the appropriate layer metadata from the fido branch (corresponding to Yocto Project 1.8):

```
$ export PATH=${PATH}:~/bin
$ mkdir ~/fsl-community-bsp
$ cd ~/fsl-community-bsp
$ repo init -u https://github.com/Freescale/fsl-community-bsp-platform -b fido
$ repo sync
```

4) Configuration

Before starting the build, Yocto requires the environment to be set according to the target form. In this example, we want to build for Nitrogen6X:



 `$ cd ~/fsl-community-bsp
$ MACHINE=nitrogen6x . setup-environment build`

5) Build

The following is the most time-consuming part. We run *bitbake*, a make-like build tool which focuses on package cross-compilation for embedded Linux distributions. The recipes included by the core-image-minimal image will be parsed and their equivalent package will be downloaded, built, and deployed to a separate directory representing our resulting root filesystem:

```
$ bitbake core-image-minimal
```

6) Deployment

Once the image is built successfully, there are several target images that are built by default. One of these is an image suitable for loading directly into an SD card. It contains all of the required binaries (bootloader, kernel, filesystem) conveniently laid out in a pre-formatted binary image.

You can find the image in the *fsl-community-bsp/build/tmp/deploy/images/nitrogen6x/* directory, ready to be flashed on our SD Card (*/dev/sdb* in our case):

```
$ sudo dd if=core-image-minimal-nitrogen6x.sdcard of=/dev/sdb
```

This is all it takes to get an embedded Linux distribution running on a target platform.

6 Things to Consider to Level up Your Linux

Witekio's Cookies Policy (<https://witekio.com/privacy-policy/>) - Privacy Policy (<https://witekio.com/privacy-policy/>) - Legal notices (<https://witekio.com/legal-notices/>) -



The above should prove that getting an embedded Linux platform up and running is not a daunting task anymore. There are of course further things to consider during product development, which require more specific knowledge. Fortunately, it is not required nor too time-consuming for many solutions.

1) Driver development

When designing hardware, one has to be aware of the equivalent software support. As Linux is the standard choice of silicon vendors for testing their own reference designs, there is a strong chance device drivers will be available for all chosen components. When replacing a particular part in the system with another one, checking whether a driver exists first is strongly recommended – reducing BoM cost is understandable, but the amount of time required to enable functionality for a non-supported hardware module should not be underestimated.

2) BSP adaptation

When choosing a System-on-Module, BSP development can be greatly simplified, lowering overall time-to-market. Whether you choose this path or roll your own, developers will have to update their kernel configuration to match the underlying hardware. The recommended way for this task is to start from a known similar hardware configuration and adapt it accordingly; the Linux kernel, for most architectures such as ARM, does this through what is called the device tree, a separate binary file containing the hardware description.

All peripherals and controllers need to be listed within this file and the kernel will dynamically load appropriate drivers and modules based on the selected and desired configuration. It is important to note that understanding the device tree format (DTS/DTI) can be daunting at first and requires some expertise to properly map hardware components with the appropriate and corresponding software pieces.

By default, neither the bootloader, kernel, or root file system are optimized for boot time. If your product requires a short boot time, as needed in multiple industries such as the automotive vertical, each component will need to be altered to reach this requirement. Multiple techniques exist for this purpose and usually involve various tricks to be used together to meet this goal, such as using u-boot's Falcon mode, deferring driver init calls, parallelizing boot scripts, etc.

4) Power consumption optimization

Power management and power consumption optimization are not tasks to be considered lightly, especially for battery-operated devices such as handhelds. By default, the Linux kernel provided by silicon vendors may not even reach the System-on-Chip lowest possible power state, requiring very low-level development in order to manage to shutdown particular components of the SoC (eventually including the cores themselves) using clock gating and power domain hardware mechanisms. This complex work is required for the system to truly go into a deep suspend state. Furthermore, the hardware needs to be carefully designed in order to prevent peripherals from being powered up at all times and be properly software-controlled. This subject is quite broad and should not be underestimated – battery longevity is one of the most important factors for end-users and unfortunately very often considered an after-thought during product development.

5) Real-time capabilities

Embedded Linux is not a real-time operating system by default. Depending on your product needs, the soft real-time configuration of the Linux kernel may be satisfactory; running Linux in a hard real-time environment is also possible through the use of third-party components, such as Xenomai, a framework that provides a pervasive, interface-agnostic, hard real-time support to user space applications. Integration of such components is not straightforward and will likely require modifications of existing drivers which applications rely on in order to make them real-time capable.



Whether it is via a simple USB thumb drive or over the air, system updates can also be a key product feature. Linux unfortunately does not have any standard mechanism for such as task, unlike other higher-level operating systems such as Android. This process typically requires modifications at multiple levels, usually involving bootloader modifications and dedicated user-space scripts or applications, and could handle bootloader, kernel, application, or entire root file system updates. Various implementations exist in the industry with the ultimate goal of maintaining device integrity and avoiding bricking due to potential data corruption or power loss during updates – implementations could be a simple ping-pong mechanism with two distinct partitions being updated every other time, or go as far as having individual partitions for each software component including a recovery or factory image for more extreme scenarios. Companies such as Witekio have ready-to-use solutions available that can be leveraged for multiple kinds of hardware configurations.

This article should have hopefully demonstrated that even though specific product customization is sometimes a must, strong Linux expertise is not required anymore to initially get an embedded Linux system up and running quickly. One of the major advantages that was not highlighted throughout this document is the similarity between embedded Linux and desktop Linux application programming which vastly helps in order to reach faster time-to-market. Applications based on cross-platform frameworks such as Qt or cloud connectivity solutions based on ecosystems such as NodeJS will often need no single modification whatsoever before being able to be deployed on an embedded target, allowing companies to focus on their actual IP from a user-friendly and familiar environment. Most development tasks can be achieved, validated, and sometimes completed on a regular desktop PC while hardware design is happening in parallel, which is now a reality thanks to the use of advanced build systems such as Yocto making this transition easier than ever in the past.

